

SIMULADOR DE GESTOR DE PROCESOS

Implementación en C
Proyecto de Semestre

Materia: Sistemas Operativos

Profesor: DR. Dante Adolfo Munoz Quintero

Semestre: Sexto Semestre 2026

Integrantes del equipo:

Netro Alvarez Julia Estefania

Del Angel Del Angel Dulce Maria

Rodas Yañez Exal Eliel

Mayo 2026

1. Introducción

Los sistemas operativos modernos gestionan múltiples procesos de forma concurrente, coordinando el acceso a recursos limitados como CPU y memoria. Comprender estos mecanismos de forma práctica es fundamental en la formación de un ingeniero en sistemas computacionales.

El presente proyecto desarrolla un simulador de gestor de procesos implementado en el lenguaje de programación C, aprovechando las primitivas de sincronización POSIX (pthreads, mutexes, variables de condición) para modelar el comportamiento real de un sistema operativo al administrar procesos, asignar recursos, aplicar algoritmos de planificación y gestionar la comunicación entre procesos.

La interfaz del simulador es una aplicación web conectada en tiempo real al programa en C mediante un servidor HTTP embebido basado en la librería Mongoose, lo que permite visualizar el estado del sistema de forma dinámica desde cualquier navegador.

Este documento se organiza de la siguiente manera: la Sección 2 presenta los objetivos del proyecto; la Sección 3 describe el diseño del sistema; la Sección 4 contiene el código fuente comentado organizado por módulo; la Sección 5 muestra capturas de la ejecución del simulador; finalmente las Secciones 6 y 7 presentan conclusiones y referencias.

2. Objetivos

2.1 Objetivo General

Desarrollar un simulador funcional de gestor de procesos en C que implemente los conceptos fundamentales de sistemas operativos, incluyendo operaciones sobre procesos, asignación de recursos, algoritmos de planificación, comunicación, sincronización y terminación de procesos, con una interfaz web.

2.2 Objetivos Específicos

- Implementar la estructura de datos PCB (Process Control Block) con PID único, estado, prioridad y lista de recursos asignados.
- Modelar recursos limitados (1 CPU, 4 GB de memoria) con solicitud, liberación y detección de conflictos.
- Implementar los algoritmos de planificación FCFS, SJF, Round Robin (quantum configurable) y por prioridades.
- Implementar mecanismos de comunicación y sincronización mediante semáforos POSIX simulados con Mutex y variables de condición.
- Demostrar sincronización mediante el problema clásico del productor-consumidor.
- Registrar causas de terminación (normal, error, interbloqueo, usuario) en un log de eventos.
- Proveer una interfaz web interactiva conectada en tiempo real al simulador que permita crear, listar, suspender y terminar procesos.

3. Diseno del Sistema

3.1 Arquitectura General

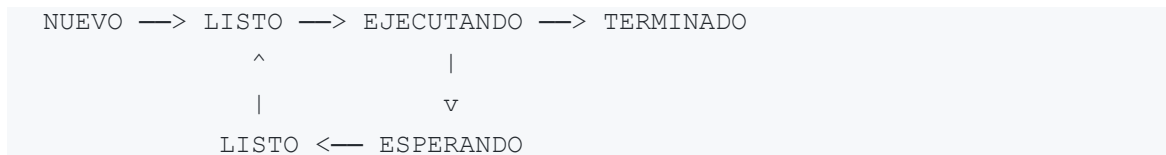
El simulador sigue una arquitectura modular de cuatro capas: núcleo (core), comunicación entre procesos (ipc), servidor HTTP (server) e interfaz web (web). Cada capa se implementa como un módulo C independiente con su propio archivo de cabecera.

Modulo	Archivo	Responsabilidad
PCB y estados	src/process.c	PCB, maquina de estados, PID atomico
Planificacion	src/scheduler.c	Cola de listos, despacho por algoritmo
Recursos	src/resource.c	Pool CPU/memoria, solicitud y liberacion
IPC	src/ipc.c	Semaforos, productor-consumidor
Logger	src/logger.c	Registro de eventos con timestamp
Servidor HTTP	src/server.c	API REST con Mongoose, sirve interfaz web
Interfaz web	web/index.html	Dashboard en tiempo real (HTML/CSS/JS)
Principal	src/main.c	Punto de entrada, orquestador

Tabla 1. Módulos del simulador y sus responsabilidades.

3.2 Diagrama de Estados del Proceso

Cada proceso transita por cinco estados posibles a lo largo de su ciclo de vida. Las transiciones son manejadas exclusivamente por la función proceso_transicion() que valida que el cambio sea legal antes de actualizarlo en el PCB.



3.3 Flujo de Datos entre Módulos

El siguiente diagrama describe como circula la información en el sistema durante el ciclo de vida de un proceso: desde que el usuario solicita su creación a través de la interfaz web hasta que el logger registra su terminación.

```

Interfaz Web (navegador)
  | HTTP POST /api/crear
  v
src/server.c —> process_crear() —> PCB en heap
  |
  | scheduler_encolar() —> Cola de listos
  |
  | recursos_solicitar() —> ResourcePool
  |
  | logger_registrar() —> Log de eventos
  |
GET /api/estado <— generar_json_estado()

```

3.4 Decisiones de Diseño Relevantes

- PID atómico: Se usa un contador global estático (`static int siguiente_pid`) con acceso controlado para garantizar unicidad sin necesidad de mutex adicional.
- Estado como enum: `ProcessState` se modela como enum de C; el compilador fuerza el uso de switch exhaustivo, reduciendo estados no manejados.
- Recursos con validacion: El `ResourcePool` valida disponibilidad antes de asignar, evitando sobreasignacion de memoria.
- Semaforo sobre Mutex+Condvar: El semaforo contador se construye sobre `pthread_mutex_t` y `pthread_cond_t`, el patron idiomático de POSIX.
- Servidor HTTP embebido: Se usa `Mongoose` (archivo unico) para evitar dependencias externas y servir la interfaz web directamente desde el ejecutable.
- Logger desacoplado: Los eventos se registran en una lista enlazada en memoria y se serializan a JSON bajo demanda, sin bloquear el nucleo.

4. Implementacion

A continuacion se presentan los fragmentos mas relevantes del codigo fuente de cada modulo. El codigo completo se encuentra en el repositorio GitHub indicado en la portada.

4.1 Operaciones sobre Procesos (PCB) — process.h / process.c

Define la estructura PCB y el enum EstadoProceso. La funcion proceso_transicion() valida las transiciones legales y actualiza el estado interno.

```
typedef enum { NUEVO, LISTO, EJECUTANDO, ESPERANDO, TERMINADO }
EstadoProceso;

typedef struct {
    int          pid;
    char          nombre[64];
    EstadoProceso estado;
    int          prioridad;    // 0 = mayor prioridad
    int          rafaga_cpu;   // rafaga estimada en ms
    int          memoria_mb;   // memoria solicitada en MB
    time_t        creado_en;
} PCB;

int proceso_transicion(PCB* pcb, EstadoProceso nuevo_estado) {
    int valida = 0;
    switch (pcb->estado) {
        case NUEVO:          valida = (nuevo_estado == LISTO);          break;
        case LISTO:          valida = (nuevo_estado == EJECUTANDO); break;
        case EJECUTANDO:     valida = (nuevo_estado == LISTO ||
                                      nuevo_estado == ESPERANDO ||
                                      nuevo_estado == TERMINADO); break;
        case ESPERANDO:     valida = (nuevo_estado == LISTO);          break;
    }
    if (valida) { pcb->estado = nuevo_estado; return 0; }
    return -1;
}
```

4.2 Asignacion de Recursos — resource.h / resource.c

Modela un pool con 1 nucleo de CPU y 4096 MB de memoria. Las solicitudes verifican disponibilidad antes de asignar.

```
typedef struct {
    int cpu_cores_libres;    // maximo 1
    int memoria_libre_mb;    // maximo 4096
}
```

```

} RecursoPool;

int recursos_solicitar(RecursoPool* pool, int cpu, int memoria_mb) {
    if (pool->memoria_libre_mb < memoria_mb) return -1;
    pool->memoria_libre_mb -= memoria_mb;
    pool->cpu_cores_libres -= cpu;
    return 0;
}

```

4.3 Algoritmos de Planificacion — scheduler.h / scheduler.c

Implementa los cuatro algoritmos requeridos. La cola de listos se mantiene como una lista enlazada. La funcion scheduler_siguiete() devuelve el proceso a despachar segun el algoritmo activo.

```

typedef enum { FCFS, SJF, ROUND_ROBIN, PRIORIDAD } Algoritmo;

PCB* scheduler_siguiete(Scheduler* s) {
    NodoCola* elegido = s->frente;
    if (s->algoritmo == SJF || s->algoritmo == PRIORIDAD) {
        // Busca el minimo en toda la cola
        NodoCola* actual = s->frente->siguiete;
        while (actual) {
            int mejor = (s->algoritmo == SJF)
                ? actual->pcb->rafaga_cpu < elegido->pcb->rafaga_cpu
                : actual->pcb->prioridad < elegido->pcb->prioridad;
            if (mejor) elegido = actual;
            actual = actual->siguiete;
        }
    }
    // FCFS y RR: simplemente extrae el frente
    return extraer_nodo(s, elegido);
}

```

4.4 Comunicacion y Sincronizacion — ipc.h / ipc.c

Implementa semaforos contadores sobre pthread_mutex_t y pthread_cond_t, y el problema del productor-consumidor con un buffer circular de capacidad 3.

```

typedef struct {
    int          valor;
    pthread_mutex_t mutex;
    pthread_cond_t  condicion;
} Semaforo;

```

```

void semaforo_wait(Semaforo* sem) {
    pthread_mutex_lock(&sem->mutex);
    while (sem->valor == 0)
        pthread_cond_wait(&sem->condicion, &sem->mutex);
    sem->valor--;
    pthread_mutex_unlock(&sem->mutex);
}

void semaforo_signal(Semaforo* sem) {
    pthread_mutex_lock(&sem->mutex);
    sem->valor++;
    pthread_cond_signal(&sem->condicion);
    pthread_mutex_unlock(&sem->mutex);
}

```

4.5 Terminacion de Procesos

El enum CausaTerminacion captura las cuatro causas de terminacion. Al terminar un proceso se liberan sus recursos y se genera una entrada en el log.

```

typedef enum {
    CAUSA_NORMAL, CAUSA_ERROR, CAUSA_INTERBLOQUEO, CAUSA_USUARIO
} CausaTerminacion;

void proceso_terminar(PCB* pcb, CausaTerminacion causa) {
    pcb->causa_fin = causa;
    proceso_transicion(pcb, TERMINADO);
}

```

4.6 Interfaz de Usuario — server.c / web/index.html

El servidor HTTP embebido con Mongoose expone una API REST con los siguientes endpoints:

Metodo	Endpoint	Descripcion
GET	/api/estado	Devuelve JSON con todos los procesos, recursos y log
POST	/api/crear	Crea un nuevo proceso con nombre, prioridad, rafaga y memoria
POST	/api/despachar	Despacha el siguiente proceso de la cola a CPU
POST	/api/terminar	Termina un proceso por PID

POST	/api/suspender	Suspende un proceso en EJECUTANDO a ESPERANDO
POST	/api/reanudar	Reanuda un proceso en ESPERANDO a LISTO
POST	/api/algorithm	Cambia el algoritmo de planificacion activo
GET	/	Sirve la interfaz web (web/index.html)

Tabla 2. Endpoints de la API REST del simulador.

5. Demostración Funcional

Las siguientes secciones describen los escenarios de prueba ejecutados en el simulador. Las capturas de pantalla correspondientes se encuentran en la carpeta capturas/ del repositorio.

5.1 Creación, Suspensión y Terminación de Procesos

Se crean tres procesos con diferentes nombres, prioridades y requerimientos de memoria. Se verifica que el sistema asigna PIDs únicos, descuenta la memoria del pool de recursos y encola cada proceso en estado LISTO. Posteriormente se suspende un proceso activo a estado ESPERANDO y se termina otro por PID, verificando la liberación correcta de recursos.

5.2 Planificación con Cada Algoritmo

Se crean varios procesos con distintas ráfagas y prioridades, y se prueba cada algoritmo de planificación:

- FCFS: los procesos se despachan en el orden en que fueron creados.
- SJF: el proceso con menor ráfaga de CPU es seleccionado primero.
- Round Robin: cada proceso recibe un quantum configurable; al agotarlo es reencolado al final.
- Prioridad: el proceso con menor número de prioridad (0 = mayor urgencia) es seleccionado primero.

5.3 Caso Productor-Consumidor (IPC)

Se ejecuta la demo del productor-consumidor con un buffer de capacidad 3, dos productores y un consumidor sincronizados con semáforos. Se verifica que el productor se bloquea cuando el buffer está lleno y el consumidor se bloquea cuando está vacío, sin condiciones de carrera.

5.4 Logs de Eventos del Simulador

El panel de registro de eventos muestra cronológicamente todas las acciones del simulador: creación de procesos, cambios de estado, asignación y liberación de recursos, cambios de algoritmo y terminaciones. Cada entrada incluye el número de evento y el PID del proceso involucrado.

Capturas de pantalla del simulador en funcionamiento

The image displays two screenshots of a web-based process simulator interface, titled "SIMULADOR DE GESTOR DE PROCESOS" (Process Manager Simulator). The interface is designed to simulate operating system processes and scheduling algorithms.

Top Screenshot:

- PROCESOS ACTIVOS (Active Processes):** Lists four processes: PID 1 - cmd, PID 2 - navegador, PID 3 - base de datos, and PID 4 - spotify. Each process shows its priority, page size, and memory usage. PID 4 is marked as "EJECUTANDO" (Running).
- CONTROL:** A central panel for managing processes. It includes a dropdown for the scheduling algorithm (currently "Round Robin"), a quantum time setting (4 ms), and buttons for "CAMBIAR ALGORITMO" (Change Algorithm), "CREAR NUEVO PROCESO" (Create New Process), "TERMINAR PROCESO" (Terminate Process), and "SUSPENDER / REANUDAR PROCESO" (Suspend / Resume Process).
- RECURSOS DEL SISTEMA (System Resources):** Displays system status: CPU at 0 / 1 libre, Memory RAM at 448 / 4096 MB, and a total of 4 processes.
- REGISTRO DE EVENTOS (Event Log):** A scrollable log showing system events, such as process creation, scheduling, and termination.

Bottom Screenshot:

- PROCESOS ACTIVOS:** Shows the same four processes, but now all are marked as "LISTO" (Ready). PID 4 is no longer running.
- CONTROL:** The interface remains the same, with the "CREAR NUEVO PROCESO" button highlighted in green.
- RECURSOS DEL SISTEMA:** The system status is updated: CPU is now 1 / 1 libre, and the total number of processes is 4.

WhatsApp x Simulador de Gestor de Proce... +

localhost:3000

SIMULADOR DE GESTOR DE PROCESOS

Sistemas Operativos · Universidad Autónoma de Tamaulipas · 2026

PROCESOS ACTIVOS

PID 1 - render_video **EJECUTANDO**

Prior: 5 Ráfaga: 2ms Mem: 1500MB

PID 2 - calculadora **LISTO**

Prior: 1 Ráfaga: 2ms Mem: 128MB

PID 3 - bloc_notas **LISTO**

Prior: 2 Ráfaga: 4ms Mem: 256MB

PID 4 - descarga **LISTO**

Prior: 3 Ráfaga: 8ms Mem: 512MB

Cola de listos: **3 procesos**

CONTROL

FCFS

Cambiar algoritmo

FCFS

Quantum (ms) - solo Round Robin

4

CAMBIAR ALGORITMO

Crear nuevo proceso

Nombre Prioridad (0-9)

editor 3

Ráfaga CPU (ms) Memoria (MB)

8 512

+ CREAR PROCESO

Terminar proceso por PID

PID

TERMINAR PROCESO

Suspender / Reanudar proceso por PID

PID

SUSPENDER PROCESO

RECURSOS DEL SISTEMA

CPU 0 / 1 libre

Memoria RAM 1700 / 4096 MB

Estado: **Proceso en CPU**

Total procesos: **4**

29°C Mayorm. soleado

Buscar

17:32 17/5/2026

WhatsApp x Simulador de Gestor de Proce... +

localhost:3000

SIMULADOR DE GESTOR DE PROCESOS

Sistemas Operativos · Universidad Autónoma de Tamaulipas · 2026

PROCESOS ACTIVOS

PID 1 - render_video **EJECUTANDO**

Prior: 5 Ráfaga: 2ms Mem: 1500MB

PID 2 - calculadora **LISTO**

Prior: 1 Ráfaga: 2ms Mem: 128MB

PID 3 - bloc_notas **LISTO**

Prior: 2 Ráfaga: 4ms Mem: 256MB

PID 4 - descarga **LISTO**

Prior: 3 Ráfaga: 8ms Mem: 512MB

Cola de listos: **3 procesos**

CONTROL

FCFS

Cambiar algoritmo

FCFS

Quantum (ms) - solo Round Robin

4

CAMBIAR ALGORITMO

Crear nuevo proceso

Nombre Prioridad (0-9)

editor 3

Ráfaga CPU (ms) Memoria (MB)

8 512

+ CREAR PROCESO

Terminar proceso por PID

PID

TERMINAR PROCESO

Suspender / Reanudar proceso por PID

PID

SUSPENDER PROCESO

RECURSOS DEL SISTEMA

CPU 0 / 1 libre

Memoria RAM 1700 / 4096 MB

Estado: **Proceso en CPU**

Total procesos: **4**

29°C Mayorm. soleado

Buscar

17:33 17/5/2026

WhatsApp x Simulador de Gestor de Proce... +

localhost:3000

PROCESOS ACTIVOS

PID 1 - render_video
Prior: 3 Báfaga: 25ms Mem: 1500MB
ESPERANDO

PID 2 - calculadora
Prior: 1 Báfaga: 2ms Mem: 128MB
EJECUTANDO

PID 3 - bloc_notas
Prior: 2 Báfaga: 4ms Mem: 250MB
LISTO

PID 4 - descarga
Prior: 3 Báfaga: 8ms Mem: 512MB
LISTO

Cola de listos: 2 procesos

CONTROL

FCFS

Cambiar algoritmo
FCFS

Quantum (ms) - solo Round Robin
4

CAMBIAR ALGORITMO

Crear nuevo proceso
Nombre: editor Prioridad (0-9): 3
Báfaga CPU (ms): 8 Memoria (MB): 512
+ CREAR PROCESO

Terminar proceso por PID
PID
TERMINAR PROCESO

Suspender / Reanudar proceso por PID
PID
SUSPENDER PROCESO
REANUDAR PROCESO
DESFACHAR SIGUIENTE

RECURSOS DEL SISTEMA

CPU 0 / 1 libre

Memoria RAM 1700 / 4096 MB

Estado: Proceso en CPU

Total procesos: 4

29°C Mayorm. soleado 17:33 17/5/2026

WhatsApp x Simulador de Gestor de Proce... + Procesos para Probar Round R...

localhost:3000

PROCESOS ACTIVOS

No hay procesos creados

Cola de listos: 0 procesos

CONTROL

SJF

Cambiar algoritmo
SJF

Quantum (ms) - solo Round Robin
4

CAMBIAR ALGORITMO

Crear nuevo proceso
Nombre: editor Prioridad (0-9): 3
Báfaga CPU (ms): 8 Memoria (MB): 512
+ CREAR PROCESO

Terminar proceso por PID
PID
TERMINAR PROCESO

Suspender / Reanudar proceso por PID
PID
SUSPENDER PROCESO
REANUDAR PROCESO
DESFACHAR SIGUIENTE

RECURSOS DEL SISTEMA

CPU 1 / 1 libre

Memoria RAM 4096 / 4096 MB

Estado: Idle

Total procesos: 0

WhatsApp x Simulador de Gestor de Proce... x Procesos para Probar Round R... x +

localhost:3000

PROCESOS ACTIVOS

No hay procesos creados

Cola de listos: 0 procesos

CONTROL

SJF

Cambiar algoritmo

SJF

Quantum (ms) - solo Round Robin

4

CAMBIAR ALGORITMO

Crear nuevo proceso

Nombre Prioridad (0-9)

editor 3

Ráfaga CPU (ms) Memoria (MB)

8 512

+ CREAR PROCESO

Terminar proceso por PID

PID

TERMINAR PROCESO

Suspender / Reanudar proceso por PID

PID

SUSPENDER PROCESO

▶ REANUDAR PROCESO

▶ DESPACHAR SIGUIENTE

RECURSOS DEL SISTEMA

CPU 1 / 1 libre

Memoria RAM 4096 / 4096 MB

Estado: idle

Total procesos: 0

29°C Mayorm. soleado 17:34 17/5/2026

WhatsApp x Simulador de Gestor de Proce... x Procesos para Probar Round R... x +

localhost:3000

PROCESOS ACTIVOS

PID 1 - word LISTO

Prior: 3 Ráfaga: 6ms Mem: 256MB

PID 2 - antivirus LISTO

Prior: 4 Ráfaga: 10ms Mem: 1024MB

PID 3 - notificacion EJECUTANDO

Prior: 1 Ráfaga: 2ms Mem: 64MB

PID 4 - reproductor LISTO

Prior: 2 Ráfaga: 4ms Mem: 512MB

Cola de listos: 3 procesos

CONTROL

SJF

Cambiar algoritmo

SJF

Quantum (ms) - solo Round Robin

4

CAMBIAR ALGORITMO

Crear nuevo proceso

Nombre Prioridad (0-9)

editor 2

Ráfaga CPU (ms) Memoria (MB)

4 512

+ CREAR PROCESO

Terminar proceso por PID

PID

TERMINAR PROCESO

Suspender / Reanudar proceso por PID

PID

SUSPENDER PROCESO

▶ REANUDAR PROCESO

▶ DESPACHAR SIGUIENTE

RECURSOS DEL SISTEMA

CPU 0 / 1 libre

Memoria RAM 2240 / 4096 MB

Estado: Proceso en CPU

Total procesos: 4

29°C Mayorm. soleado 17:37 17/5/2026

WhatsApp x Simulador de Gestor de Proce... +

localhost:3000

PROCESOS ACTIVOS

PID 1 - word
Prior: 3 Báfaga: 6ms Mem: 256MB LISTO

PID 2 - antivirus
Prior: 4 Báfaga: 10ms Mem: 1024MB LISTO

PID 3 - notificacion
Prior: 1 Báfaga: 2ms Mem: 64MB EJECUTANDO

PID 4 - reproductor
Prior: 2 Báfaga: 4ms Mem: 512MB LISTO

Cola de listos: 3 procesos

CONTROL

SJF

Cambiar algoritmo
SJF

Quantum (ms) - solo Round Robin
4

CAMBIAR ALGORITMO

Crear nuevo proceso
Nombre Prioridad (0-9)
editor 2
Báfaga CPU (ms) Memoria (MB)
4 512

+ CREAR PROCESO

Terminar proceso por PID
PID

TERMINAR PROCESO

Suspender / Reanudar proceso por PID
PID

SUSPENDER PROCESO

REANUDAR PROCESO

DESFACHAR SIGUIENTE

RECURSOS DEL SISTEMA

CPU 0 / 1 libre

Memoria RAM 2240 / 4096 MB

Estado: Proceso en CPU

Total procesos: 4

29°C Mayorm. soleado

ESP LAA

17:37 17/5/2026

WhatsApp x Simulador de Gestor de Proce... +

localhost:3000

PROCESOS ACTIVOS

PID 1 - word
Prior: 3 Báfaga: 6ms Mem: 256MB LISTO

PID 2 - antivirus
Prior: 4 Báfaga: 10ms Mem: 1024MB LISTO

PID 3 - notificacion
Prior: 1 Báfaga: 2ms Mem: 64MB ESPERANDO

PID 4 - reproductor
Prior: 2 Báfaga: 4ms Mem: 512MB LISTO

Cola de listos: 3 procesos

CONTROL

SJF

Cambiar algoritmo
SJF

Quantum (ms) - solo Round Robin
4

CAMBIAR ALGORITMO

Crear nuevo proceso
Nombre Prioridad (0-9)
editor 2
Báfaga CPU (ms) Memoria (MB)
4 512

+ CREAR PROCESO

Terminar proceso por PID
PID

TERMINAR PROCESO

Suspender / Reanudar proceso por PID
PID

SUSPENDER PROCESO

REANUDAR PROCESO

DESFACHAR SIGUIENTE

RECURSOS DEL SISTEMA

CPU 1 / 1 libre

Memoria RAM 2240 / 4096 MB

Estado: Idle

Total procesos: 4

29°C Mayorm. soleado

ESP LAA

17:37 17/5/2026

WhatsApp x Simulador de Gestor de Proce... x Procesos para Probar Round R... +

localhost:3000

Sistemas Operativos · Universidad Autónoma de Tamaulipas · 2026

PROCESOS ACTIVOS

No hay procesos creados

Cola de listos: 0 procesos

CONTROL

Prioridad

Cambiar algoritmo

Prioridad

Quantum (ms) - solo Round Robin

4

CAMBIAR ALGORITMO

Crear nuevo proceso

Nombre Prioridad (0-9)

editor 2

Ráfaga CPU (ms) Memoria (MB)

4 512

+ CREAR PROCESO

Terminar proceso por PID

PID

TERMINAR PROCESO

Suspender / Reanudar proceso por PID

PID

SUSPENDER PROCESO

▶ REANUDAR PROCESO

RECURSOS DEL SISTEMA

CPU 1 / 1 libre

Memoria RAM 4096 / 4096 MB

Estado: Idle

Total procesos: 0

29°C Mayorm. soleado 17:39 17/5/2026

WhatsApp x Simulador de Gestor de Proce... +

localhost:3000

Sistemas Operativos · Universidad Autónoma de Tamaulipas · 2026

PROCESOS ACTIVOS

PID 1 - actualizacion LISTO

Prior: 8 Ráfaga: 6ms Mem: 256MB

PID 2 - driver_audio ESPERANDO

Prior: 1 Ráfaga: 4ms Mem: 128MB

PID 3 - steam LISTO

Prior: 6 Ráfaga: 10ms Mem: 1024MB

PID 4 - discord LISTO

Prior: 3 Ráfaga: 5ms Mem: 512MB

Cola de listos: 3 procesos

CONTROL

Prioridad

Cambiar algoritmo

Prioridad

Quantum (ms) - solo Round Robin

4

CAMBIAR ALGORITMO

Crear nuevo proceso

Nombre Prioridad (0-9)

editor 3

Ráfaga CPU (ms) Memoria (MB)

5 512

+ CREAR PROCESO

Terminar proceso por PID

PID

TERMINAR PROCESO

Suspender / Reanudar proceso por PID

PID

SUSPENDER PROCESO

▶ REANUDAR PROCESO

▶ DESPACHAR SIGUIENTE

RECURSOS DEL SISTEMA

CPU 1 / 1 libre

Memoria RAM 2176 / 4096 MB

Estado: Idle

Total procesos: 4

29°C Mayorm. soleado 17:41 17/5/2026

6. Conclusiones

Netro Álvarez Julia Estefania

Trabajar en los algoritmos de planificacion me permitio comprender en la practica las ventajas y limitaciones de cada uno. Implementar FCFS fue sencillo, pero al codificar SJF me di cuenta de que puede causar inanicion en procesos con rafagas largas, algo que solo habia visto en teoria. Round Robin fue el mas interesante porque el quantum cambia completamente el comportamiento del sistema: con un quantum pequeno el sistema responde rapido pero genera mucho cambio de contexto, y con uno grande se parece mas a FCFS. El algoritmo por prioridad me hizo reflexionar sobre la importancia de definir bien las politicas de un sistema operativo, ya que una mala asignacion de prioridades puede bloquear procesos importantes. Esta experiencia me dejo claro que no existe un algoritmo perfecto, sino que cada uno es optimo segun el tipo de carga de trabajo.

Del Ángel Del Ángel Dulce María

Implementar la comunicacion y sincronizacion entre procesos fue la parte mas desafiante del proyecto. Al principio el concepto de semaforo me parecia abstracto, pero al tener que construirlo desde cero con `pthread_mutex_t` y `pthread_cond_t` entendi exactamente como funciona internamente: el mutex protege el valor del semaforo y la variable de condicion es la que bloquea y despierta los hilos. El problema del productor-consumidor fue clave para entender por que necesitamos tres semaforos: uno para exclusion mutua, uno para los espacios vacios y otro para los espacios llenos. Sin ellos el buffer puede corromperse. Tambien aprendi que la sincronizacion incorrecta puede generar condiciones de carrera muy dificiles de detectar, porque no siempre se reproducen de forma consistente. Esta parte del proyecto me dio una vision muy solida de como los sistemas operativos reales coordinan procesos concurrentes.

Rodas Yanez Exal Eliel

Mi parte del proyecto fue el nucleo del simulador: el PCB, los estados del proceso y la integracion de todos los modulos a traves del servidor HTTP y la interfaz web. Definir el PCB como una estructura en C me hizo entender por que los sistemas operativos reales necesitan mantener tanta informacion de cada proceso: sin el PID, el estado, la prioridad y los recursos asignados, seria imposible administrar multiples procesos de forma ordenada. La parte mas retardora fue conectar el simulador en C con la interfaz web mediante Mongoose, ya que tuve que aprender como funciona HTTP a nivel de peticiones y respuestas para construir la API REST. Integrar todos los modulos tambien me enseno la importancia de definir interfaces claras desde el inicio: cuando los modulos tienen funciones bien definidas en sus archivos de cabecera, la integracion es mucho mas sencilla. En general, este proyecto me dio una perspectiva practica muy valiosa sobre el funcionamiento interno de un sistema operativo.

7. Referencias

- [1] Silberschatz, A., Galvin, P. B. y Gagne, G. (2018). Operating System Concepts (10.a ed.). Wiley.
- [2] Tanenbaum, A. S. y Bos, H. (2015). Modern Operating Systems (4.a ed.). Pearson.
- [3] Kerrisk, M. (2010). The Linux Programming Interface. No Starch Press.
- [4] IEEE Std 1003.1 — POSIX Threads. (2024). Recuperado de <https://pubs.opengroup.org/onlinepubs/9699919799/>
- [5] Cesanta Software. (2024). Mongoose Web Server Library. Recuperado de <https://mongoose.ws/>
- [6] Downey, A. B. (2016). The Little Book of Semaphores (2.a ed.). Green Tea Press. Recuperado de <https://greenteapress.com/semaphores/>